

Danmarks
Tekniske
Universitet



Application for Testing and Measuring multi-RAT

AUTHORS

Stefan Artur Stefirta - s244813

Vedanta Sinha - s245010

Viktor Munk - 240335

25. marts 2026

Indhold

1	Abstract	1
2	Introduction	2
3	Theory	3
3.1	Protocol Choice: UDP or QUIC	3
3.2	Quality of Service	4
3.3	Multi-RAT and 1+1 Protection	4
3.4	IEEE 802.1CB FRER (Frame Replication and Elimination)	4
3.5	Different Radio Interface characteristics	5
4	System Overview	7
4.1	Overall System concept	7
4.2	Sender	7
4.3	Receiver + FRER deduplication	9
4.3.1	Reception	9
4.3.2	FRER Deduplication	9
4.3.3	Metrics Computation	9
4.4	Web Dashboard	10
4.4.1	Backend: Express server	10
4.4.2	Receiver	10
4.4.3	Frontend: Browser client	11
4.4.4	Future Improvements	11
4.5	Preliminary results	12
4.5.1	Individual Path Performance	12
4.5.2	Merged Stream	13
4.5.3	Packet Statistics	13
5	Functional Requirements	14
5.1		14
5.2		14
5.3		14
5.4		14
6	Implementation	15
6.1	Choices of platforms	15
6.2	Protocol design	15
6.3	Tradeoffs and Limitations	15
6.4		15
7	Time Plan	16
8	Conclusion	17

9	References	18
	List of Figures	19
	List of Tables	20
	References	21

1 Abstract

.1.

2 Introduction

In transportation systems, multiple roads often connect the same cities to increase reliability and reduce congestion. This principle can also be applied to data communication. An analogy can be drawn to sending an important letter. By simultaneously delivering duplicate letters using different routes and transportation methods, the chance that at least one copy of the letter reaches the intended destination increases. Furthermore, once one letter arrives, any additional copies can simply be disregarded, unless the first one was damaged. This approach increases reliability and is often most useful when the propagation time of information from A to B is long.

Most modern wireless communication systems rely on a single type of wireless access technology, such as either WiFi, cellular RAN, or LoRaWAN, to provide connectivity. This is equivalent to connecting two cities with only a single road. Although it is often sufficient, such a setup is not optimally stable due to common challenges such as interference, congestion, mobility, and varying channel conditions. These factors can introduce increased latency, packet loss, jitter, and reduced throughput, which decreases the Quality of Service (QoS). In emergencies, it would be beneficial to implement an intelligent integration that makes the User Equipment (UE) use the most optimal combination of radio interface depending on the situation, thereby improving the overall network performance.

Multiple Radio Access Technology (Multi-RAT) refers to systems that enable communication over multiple radio access interfaces, helping ensure reliable and timely data delivery over wireless networks. In many cases, duplicate packets are transmitted using different radio access technologies simultaneously. The receiver processes the first successfully received packet and discards any duplicates. This mechanism is known as 1+1 protection.

The scope of this project is to develop an intelligent mobile application that analyzes network parameters and assists in network optimization by selecting which radio access technologies and protocols to use.

This study examines the extent to which the choice of radio access technology impacts network performance in Multi-RAT systems through analysis of metrics such as latency, reliability, hop count, and other communication parameters.

3 Theory

Each radio access technology that is compatible with Multi-RAT has its own characteristics in terms of coverage, bandwidth, latency, reliability, and access protocols. Using Multi-RAT, it allows these technologies to combine their strengths and cover their weaknesses.

This project uses Wi-Fi and LoRaWAN; LoRaWAN establishes a direct, long-range connection between the sender and the receiver. In contrast, Wi-Fi uses various access points between the sender and receiver. Although Wi-Fi has a much higher capacity than LoRaWAN, unlike LoRaWAN, it accumulates additional latency with every network hop.

This, theoretically, means that in shorter distances that consist of only a few network hops, Wi-Fi will be much more efficient and practical than LoRaWAN. However, once the number of network hops increases, a direct connection like LoRaWAN will eventually become more efficient and practical. This project will aim to find the practical measurement, where this becomes the case.

If implementing the use of mobile RAN in this project, it would theoretically have capabilities that are between the LoRa and WiFi in terms of distance and bitrate proportionality, because of the increased access range, but decreased bitrate. This is why it would be relevant to create a system that runs diagnostics on which radio access technology is the most reliable to use by not only looking at the type of service, but also at the position of the UE and utilising a combination of those two. It is important to mention that there are already "Download Booster" apps publicly available, which allow the phone to utilise both WiFi and Mobile data at the same time, to reach faster download speeds; however, none of them are intelligent, that implement different protocols and rather focus on reliability than download speeds.

3.1 Protocol Choice: UDP or QUIC

The selection of a fitting transport protocol for this communication system is vital. As established before, Multi-RAT aims to improve efficiency by mitigating the effects of packet drops and transmission errors. To complement this efficiency-based perspective, this project uses User Datagram Protocol because it does not require connection setup, acknowledgments, or retransmission mechanisms. All of this reduces the processing complexity and additional header data. This makes it a good choice for systems such as FRER.

UDP has one glaring problem. It lacks encryption, and since Multi RAT aims to transmit sensitive data quickly, it must ensure that such data is not compromised. Thus, this project uses Quick UDP Internet Connection (QUIC), which builds upon UDP by incorporating a TLS style encryption system. It also introduces improved congestion control and connection management, which also help in achieving the goals of a Multi RAT system.

3.2 Quality of Service

Radio communication is a service, and service-providing organisations continually seek to improve their services for either profit or social good. For them to improve their service, they have to know about its quantitative parameters, which are called Quality of Service (QoS). These parameters are latency, jitter, packet loss, and throughput.

Latency is the measure of time between the sender sending the packet and the receiver receiving the packet. This becomes crucial in critical scenarios, where every second counts. Jitter is the variation in the latency caused by separate delays. High jitter can be detrimental, especially for Live services. Packet Loss is the percentage of the total packets that were sent but not received, or sent but were dropped due to a failed checksum. This is critical! And one of the main issues that Multi RAT tries to combat. Finally, throughput is how much data can be sent per time interval.

3.3 Multi-RAT and 1+1 Protection

1 + 1 protection is a transmission method in which the same packet is sent over two separate communication paths at the same time. In a Multi-RAT system, this means that two different radio interfaces can be used in parallel to carry identical data. The receiver then accepts the first correctly received packet and discards any duplicate copies. This approach increases reliability, since communication can still succeed even if one path is delayed, disturbed, or temporarily unavailable. It is especially useful in applications where stable and continuous packet delivery is more important than minimizing bandwidth usage. A major strength of 1+1 protection is its ability to reduce the impact of packet loss and link failures.

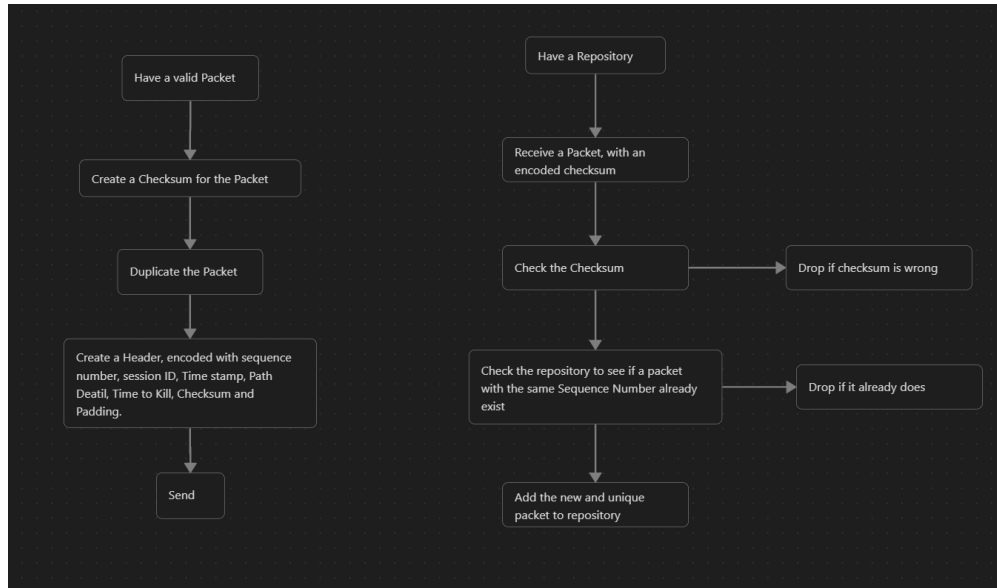
However, the method also introduces redundancy, because the same data is transmitted more than once. This increases resource consumption and may reduce overall efficiency compared to single-path transmission. But overall, 1+1 is an important mechanism in Multi-RAT systems, as it improves robustness by allowing multiple radio technologies to support the same communication flow.

3.4 IEEE 802.1CB FRER (Frame Replication and Elimination)

IEEE 802.1CB, otherwise known Frame Replication and Elimination (FRER), is the mechanism that allows for the 1+1 model to be implemented. The flow charts below show the process that is taken for FRER.

In Figure 1, the Flow chart on the left depicts the FRER functionality of the Sender. It starts with a packet, which is randomly generated in this project. To handle error detection, a checksum is computed for this packet. After computing the checksum, the packet is duplicated.

At the same moment of time, the application header is encoded. This has the sequence number, session ID, time stamp, the path detail, Time to kill, the previously calculated checksum,



Figur 1: Sender and Receiver Flowchart for FRER

and padding. These duplicate packets can now be sent simultaneously via different radio access networks.

On the right is the flowchart modeling the receiver of the FRER. The receiver's process begins with analysing the application header of the incoming packet, specifically for the checksum. It calculates the checksum of the received packet, checking to see if it matches the checksum in the application header. If the checksum is incorrect, the packet is faulty and is thus dropped.

If the packet has the correct checksum, the receiver checks an internal repository to see if another packet with the same sequence number is present. If one is, then this packet is a duplicate and is dropped. Otherwise, this packet is stored in the repository and forwarded.

3.5 Different Radio Interface characteristics

Wireless systems rely on different radio interfaces, each with its own strengths and limitations. Some are better suited for high speed data transfer and low latency, while others are designed for extended coverage, low energy consumption, or robust connectivity. Therefore, the characteristics of a radio interface play an important role on determining how packets are transmitted and how well a communication system can meet specific application requirement.

In this context, Multi-RAT stands out because it allows a device or network to use more than one radio access technology instead of relying on a single interface. This makes it possible to select the most suitable communication method based on factors such as channel conditions, traffic type, latency requirements, and power constraints. as a result, Multi-RAT can offer improved flexibility, reliability, and overall network performance. Furthermore, Multi-RAT can combine the characteristics of different radio interfaces to achieve more efficient packet

transmission by taking advantage of the strengths of each technology.

Wi-Fi/IEEE

Wi-Fi is a short to medium range wireless communication technology mainly designed for high data throughput in local area networks. It operates in the unlicensed 2.4 GHz and 5 GHz bands which explains high throughput with low latency. Furthermore, Wi-Fi is well suited for applications that require fast packet transmission, such as video streaming, file transfer, and internet access. A major strength of Wi-Fi is its wide availability, low deployment cost, and ability to support a large amount of data traffic.

On the down side Wi-Fi coverage is limited compared to cellular and long range technologies, especially in environments with walls, interference, or when many devices share the same access point.

LoRaWAN/LPWAN

LoRaWAN is low power wide area network technology designed for long range communication with low data rates and low energy consumption. It operates in sub GHz unlicensed bands and is highly resilient to interference. It is particularly suitable for IoT applications such as environmental monitoring, smart metering, and remote sensing, where devices only need to send small amounts of data occasionally, and hard to reach areas where other infrastructure may be limited.

However, LoRaWAN has clear limitations in terms of throughput, latency, and payload size. It is not appropriate for applications that require continuous data transfer, large packets, or strict real time communication. Network capacity can also become a challenge when many devices attempt to communicate frequently.

5G/LTE

5G is a cellular radio technology developed to support high data rates, low latency, massive device connectivity, and improved network reliability. It is designed for a wide range of applications, including enhanced mobile broadband and industrial automation. Its strength is the ability to deliver very high throughput and low end to end delay, making it suitable for demanding real time services. Compared to previous cellular generations, 5G also offers better support for dense deployments and advanced network management. Besides that it has broad coverage, especially when deployed on lower frequency bands, which makes it more flexible than many local wireless technologies.

However, 5G infrastructure is complex and expensive to deploy and maintain. Its performance also depends strongly on spectrum band, since higher frequency bands can provide very high capacity but have shorter range and weaker penetration.

4 System Overview

.1.

4.1 Overall System concept

The overall system focuses on sending 2 identical packets that are randomly generated, symbolizing the critical information that has to be received.

4.2 Sender

The sender is implemented in python and is responsible for generating and transmitting duplicate packets simultaneously over two separate UDP paths, realizing the 1+1 protection scheme described in Section 3.3.

At startup, the sender generates a random 32 bit session ID using a cryptographically secure random number generator. This session ID remains constant throughout the entire transmission session and is embedded in every packet, allowing the receiver to distinguish packets belonging to different sessions, for example when the sender is restarted.

For each packet, the sender constructs a custom application header as illustrated in Figure 2. The header consists of seven fields totaling 21 bytes:

- **Sequence Number (4 bytes):** A monotonically incrementing counter used by the receiver to detect packet loss and identify duplicates.
- **Session ID (4 bytes):** The random generated session identifier.
- **Timestamp (8 bytes):** The current system time in nanoseconds at the moment of transmission, used by receiver to compute one way latency.
- **Path ID (1 byte):** Identifies which radio path the packet was sent on e.g. 1 or 2.
- **Sent TTL (1 byte):** The IP Time To Live value set on the socket at send time, embedded in the header so the receiver can calculate the number of network hops traversed.
- **CRC16 (2 bytes):** A CRC checksum computed over the payload, used by the receiver for integrity validation.
- **Pad (1 byte):** Padding for header alignment.

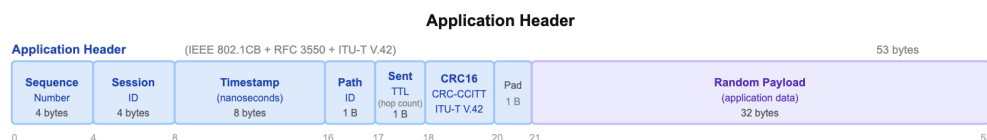
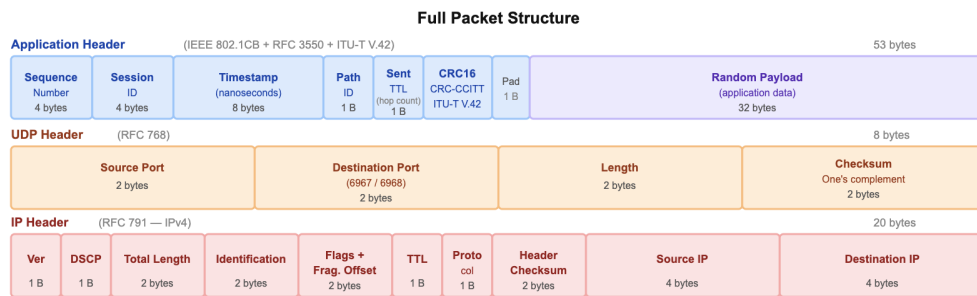


Figure 2: Our Custom built Application Header

A random 32 byte payload is appended after the header, resulting in a total packet size of 53 bytes as shown in the Figure 3. The full packet is then encapsulated in a UDP datagram,

which is itself carried in an IPv4, adding a further 8 bytes of UDP header and 20 bytes of IP header.

The sender transmits packets at a configurable rate, 20 packets per second, by calculating the precise time of each transmission and sleeping until the next scheduled send. For each sequence number, two identical packets are constructed, one with path ID set to 1 and the other 2, sent to 2 different ports, and dispatched simultaneously, ensuring both paths carry the same data at the same instant.



Figur 3: The full packet structure

Figure 3 illustrates the complete packet structure as it appears on the network. The packet is composed of three layers of headers, each serving a distinct purpose in the transmission process, reflecting the layered nature of the Internet protocol stack.

The **Application Header** is the outermost layer in terms of design responsibility, it is constructed entirely by the sender and carries the information needed for the receiver to perform latency measurements, FRER duplication, hop counting, and integrity validation.

Below the application header sits the **UDP Header**, which is automatically added by the operating system when the sender calls `sendto()`. It consists of 8 bytes containing the source port, destination port, length, and a checksum. UDP was chosen as the transport protocol because it is connectionless and introduces minimal overhead, meaning packets are sent immediately without the handshaking, retransmission, and congestion control measurements, as TCP would silently retransmit lost packets and obscure the true characteristics of each radio path.

At the bottom sits the **IP Header**, added by the operating system's network stack. It is 20 bytes and contains fields such as the source and destination IP addresses, the TTL, and a header checksum. The TTL field is of particular interest to this project, the sender sets it to 64 at the IP layer via a socket option, and also embeds this value in the application header. By comparing the sent TTL with the TTL observed at the receiver, the number of routers the packet traversed can be inferred, providing a hop count metric for each path.

4.3 Receiver + FRER deduplication

The receiver is responsible for capturing incoming packets from both radio paths, validating their integrity, eliminating duplicates, and computing per path quality metrics.

4.3.1 Reception

Each receiver thread is bound to dedicated UDP port, and runs independently, ensuring that a delay or loss on one path does not affect the processing of the other. Upon receiving a packet, the thread unpacks the custom application header containing the sequence number, session ID, timestamp, path identifier, sent TTL, and checksum. The first validation step is considered corrupted and discarded, and the CRC error counter for corresponding path is incremented.

On platforms that support it, the receiver also extracts the received IP TTL from the ancillary data of the incoming packet using `recvmsg`. By comparing the sent TTL embedded in the application header with the received TTL from IP header, the number of network hops the packet traversed can be calculated as:

$$\text{hops} = \text{TTL}_{\text{Sent}} - \text{TTL}_{\text{IP-Header}}$$

Obviously this becomes more relevant when we have the radio interface 5G integrated, since the mobile technology is known for running through multiple receivers to reach destination.

4.3.2 FRER Deduplication

Once a packet passes the integrity check, it is subject to the FRER deduplication mechanism. The receiver maintains a sliding window of recently seen session ID's, pairs, with a window size of 2000 entries. A companion hash set enables $O(1)$ lookup, while a deque handles eviction of the oldest entry when the window is full. If the incoming session ID is already present in the set, the packet is identified as a duplicate and dropped, its path duplication counter is incremented, but it is not delivered to the merged stream. If the pair is new, it is added to the window and the packet is forwarded to the merged stream for further processing. This ensures that regardless of which path delivers a packet first, the application layer receiver each sequence number exactly once.

4.3.3 Metrics Computation

For every valid packet including duplication before elimination per path metrics are updated. Latency is computed as the difference between the current system time, and timestamp embedded in the packet header at send time. Jitter is calculated as the absolute difference between consecutive latency measurements, following the approach described in **RFC 3550**. Throughput is derived from the cumulative number of bytes received divided by the elapsed

time since the first packet arrived on that path. PAcKet loss is estimated from gaps in the sequence number stream, if the received sequence number is greater than the last seen sequence number by more than one, the difference minus one is counted as lost packets.

Every second, the aggregator thread computes one second averages of latency, jitter, and throughput for each path and for the merged stream, and forwards the results to the web dashboard.

4.4 Web Dashboard

To visualize the measure network parameters in real time, a web based dashboard has been developed. The dashboard is built on three tightly coupled components, **receiver**, **Node.js/Express** backend server and a **Browser client** frontend client. Together they form a pipeline that continuously captures raw UDP traffic, aggregates it into per second snapshots, and renders the results as live charts in the browser.

4.4.1 Backend: Express server

The backend is implemented using Express, a minimal web application framework for Node.js. Express handles two responsibilities, serving the static frontend files and exposing a REST endpoint that the python receiver calls once per second. When a metrics payload arrives at this endpoint, it is not stored server side, but immediately broadcast to all connected browser clients using Socket.IO.

Socket.IO is a library that abstracts real time, bidirectional communication between a server and its clients. It preferentially uses the WebSocket protocol, which establishes a persistent, full duplex TCP between the browser and the server. Unlike traditional HTTP, where the client must repeatedly poll the server for updates, WebSocket allows the server to push data to the client the instant it becomes available. This is essential for a live network monitor, where one second resolution is required and polling would introduce unnecessary latency and overhead. If WebSocket is unavailable, for example due to a restrictive proxy, Socket.IO falls back to automatically to HTTP long polling, ensuring broad compatibility.

The Express server also manages the lifecycle of the python receiver by spawning it as a child process on startup. This means the entire measurement system, from raw packet capture to live visualization, is started with a single command, reducing operational complexity

4.4.2 Receiver

The receiver runs two dedicated threads, one per UDP socket, that continuously capture incoming packets and update shared in memory metric structures protected by a mutex lock. A third aggregator thread wakes every second, computes one second averages for latency, jitter, throughput, and packet loss across both individual paths and the merged FRER stream, and serializes the result as a JSON payload. This payload is delivered to the Express

server via a standard HTTP POST request using python's built in **urllib** library. The request is fire and forget if the Express server is temporarily unavailable, the metric snapshot is silently discarded rather than queued, preventing unbound memory growth.

4.4.3 Frontend: Browser client

The browser client is a single HTML file that connects to the server using the Socket.IO JavaScript client. Upon receiving a metric, the client updates both numeric readouts and the live charts without reloading the page.

Charts are rendered using **Chart.js**, a widely used open source JavaScript charting library built on the HTML5 Canvas API. Each metric, latency, jitter, throughput and packet loss is represented as a time series line chart. The chart maintains a rolling windows of the 60 most recent data prints, discarding older entries as new ones arrive, so the x axis always represents the last 60 seconds of activity.

The dashboard is structured into five tabs: **Overview**, **WiFi**, **WiFi**, **Merged**, and **Packets**.

- **Overview:** The overview tab is meant to give the client full overview of the progress
- **WiFi:** We have 2 WiFi tabs since right now we only have integrated the WiFi radio interface, so 1 tab for each path through WiFi. The goal is to get the other radio interfaces integrated and give each interface/path/stream a tab each.
- **Merged:** The tab for merged streams, shows cases the merged stream from all the different paths, and takes the best from each path that way we can make the most optimal path for the packet to reach destination fastest and safest.
- **Packets:** The packet tab additionally tracks cumulative counters such as total duplicates eliminated by FRER and CRC errors detected, giving long running summary that complements the rolling charts.

4.4.4 Future Improvements

The plan is a extension of the current system and make it a dedicated mobile application to replace the browser based dashboard. While the existing web dashboard provides real time visualization of network metrics, it remains a passive monitoring tool running on a separate machine. A mobile application would move the intelligence directly onto the User Equipment (UE), enabling active participation in the Multi RAT system.

The mobile application would retain the live metric visualization of the current dashboard, plotting latency, jitter, throughput, and packet loss in real time, but would additionally control the radio interfaces of the device itself. Rather than receiving packets on two fixed UDP ports from an external sender, the mobile device would act as the sender, transmitting duplication packets simultaneously over both WiFi and 5G or just 1 of the interfaces. This would allow the application to measure the actual QoS of each radio interface from the perspective of the end user in their current location and environment.

Based on the continuously measure metrics, the application would implement an intelligent interface selection algorithm to analyses the performance of each path and recommends or automatically switch to the most suitable radio interfaced. For example, if the 5G path exhibits consistently lower latency and packet loss than WiFi, the application would notify the user and priorities 5G for subsequent transmissions. Conversely, if WiFi provides higher throughput at a given location, the system would favour it accordingly.

This would represent a shift from a passive measurement tool to an active, self optimizing Multi RAT client, which is the long term vision of the project as described in the introduction.

4.5 Preliminary results

To validate the system, a preliminary test was conducted over a local WiFi network, with the sender and receiver connected via two sepaprate WiFi paths on the same network. The results are presented on Figures 4 - 7.

4.5.1 Individual Path Performance

As shown in the Figures 4, 5 both paths exhibits similar behavior, which is expected given that both used WiFi on the same local network. Path 1 recorded an average latency of 49.36 ms with a jitter of 9.50 ms, while path path 2 recorded 48.07 ms latency and 9.72 ms jitter. Throughput was stable at 5.1 kbps on both paths, consistent with the configured packet rate of 20 packets per second at a payload size of 32 bytes. Neither path experienced any packet loss or CRC errors throughout the test, confirming the integrity of the transmission and the correctness of the checksum validation

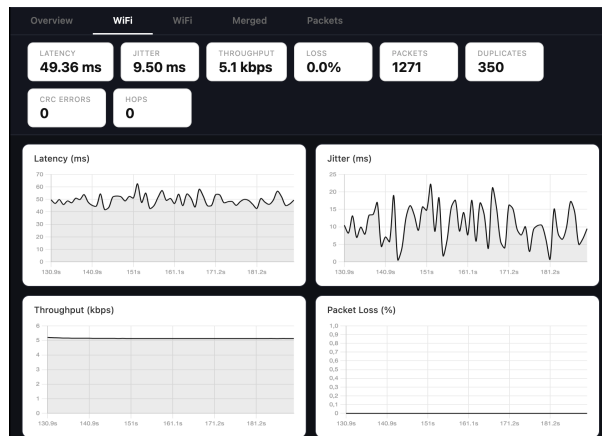


Figure 4: Data from WiFi path 1

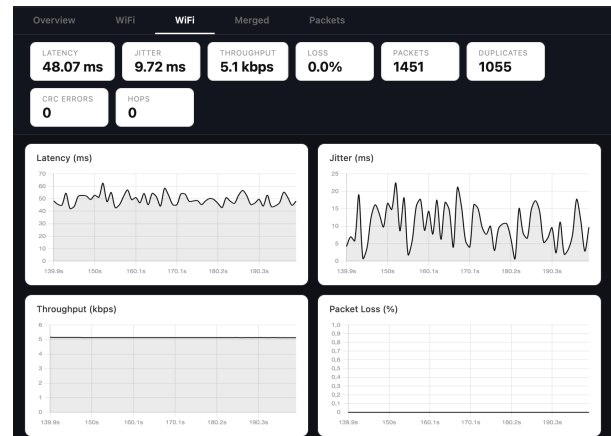


Figure 5: Data from WiFi path 2

4.5.2 Merged Stream

The merged stream, shown in Figure 6, demonstrates the effect of FRER deduplication. The average latency of the merged stream was 43.08 ms, slightly lower than either individual path, as the merged stream always accepts whichever copy arrives first. Most notably, jitter dropped significantly to 0.49 ms, compared to approximately 9.5 ms on each individual path. This reduction illustrates one of the key benefits of 1+1 protection, by selecting the first arriving packet from two independent paths, timing irregularities on either path are partially absorbed, resulting in a smoother and more consistent delivery stream.

4.5.3 Packet Statistics

The last figure 7 showcases the cumulative overview of the test. A total of 3592 packets were received across both paths, 1796 on each, which 1796 were delivered to the application layer as unique packets and 1796 were eliminated as duplicates by FRER mechanism. This confirms that the deduplication is functioning correctly, with every transmitted sequence number arriving on both paths and exactly one copy being forwarded. Zero packets were lost and zero CRC errors were detected across the entire session.



Figure 6: The merged stream from path 1 and 2

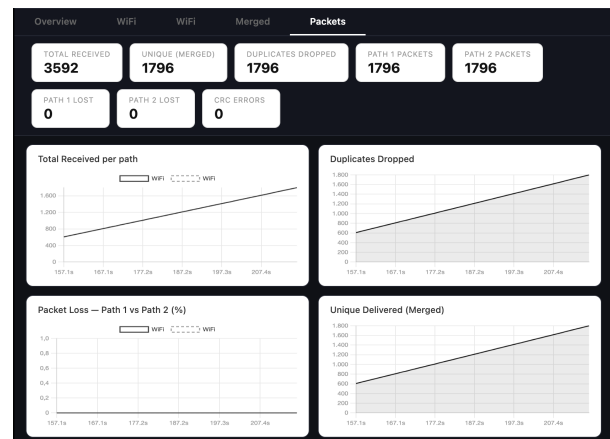


Figure 7: The overall info about packets the servers received

5 Functional Requirements

.1.

5.1

5.2

5.3

5.4

6 Implementation

.1.

6.1 Choices of platforms

6.2 Protocol design

6.3 Tradeoffs and Limitations

6.4

7 Time Plan

.1.

8 Conclusion

.1.

9 References

.1.

Figurer

1	Sender and Receiver Flowchart for FRER	5
2	Our Custom built Application Header	7
3	The full packet structure	8
4	Data from WiFi path 1	12
5	Data from WiFi path 2	12
6	The merged stream from path 1 and 2	13
7	The overall info about packets the servers received	13

Tabeller

Litteratur